

空间记忆研究：数据集选择与实验落地步骤

面向 DINO latent + 结构对齐 persistent memory + 静态/动态双记忆

目标

把 DINO-WM 的“离线轨迹 + 视觉 latent dynamics”思路扩展到真实室内动态环境：模型不仅预测下一帧 latent，而是维护一个与世界空间绑定、能够跨视角持续存在，并能区分临时遮挡与真实长期变化的空间记忆。

这份文档解决什么问题

- 选择哪些数据集分别负责原型验证、可控训练、真实环境测试与动态压力测试。
- 把不同来源的数据统一成同一种 episode 格式，避免后续每换一个数据集就重写模型。
- 给出从 DINO-WM baseline 到最终 real-world benchmark 的具体实施顺序。
- 预先定义空间记忆专属指标，而不是只看下一帧预测 loss。

推荐的总路线

0. DINO-WM 基线	1. THUD++ / Habitat 可控训练	2. ADT World Slot 验证	3. LILocBench 真实主实验	4. JRDB / Dynamic Replica 压力测试
------------------	-----------------------------	-------------------------	------------------------	--------------------------------------

1. 我们的数据到底需要什么

DINO-WM 的核心输入可以抽象为连续离线轨迹： $(\mathbf{o}_t, \mathbf{a}_t, \mathbf{o}_{t+1})$ ，并在 DINOv2 patch feature 空间中预测未来状态。我们的空间记忆版本需要进一步知道“当前观察来自世界中的哪里”，以及“变化来自机器人自身运动还是动态目标”。

统一目标字段

每个时间步尽量组织为 $X_t = \{\text{RGB, depth, camera/robot pose, action/velocity, semantic/instance annotation, dynamic mask/track}\}$ 。其中 pose + depth 是跨视角把 frame observation 写入 persistent world memory 的关键。

2. 数据集角色分工

数据集	最适合做什么	关键可用信息	在论文中的位置
DINO-WM 原实验	复现 latent world-model baseline	RGB 离线轨迹、动作；PointMaze 2000×100, Wall 1920×50, Push-T 18500 条	方法起点 / sanity baseline
THUD++	训练动态场景表示；static/dynamic 对照；仿真控制实验	13 个动态室内场景；RGB-D、pose、语义/实例/2D-3D 标注；含真实与合成，另有导航仿真器	训练 + controlled ablation
Habitat 3.0	自己生成严格可控的遮挡/转动/人群实验	可控机器人、humanoid、Social Navigation / Social Rearrangement；可生成 pose/action/GT	训练 + 因果式消融
Aria Digital Twin (ADT)	验证同一物体/表面能否跨帧落到同一 world slot	RGB、depth、6DoF trajectory、instance segmentation、object_uid、object pose；motion_type=static/dynamic	核心几何/关联验证
LILocBench	区分临时动态与长期真实变化	3×RGB-D、wheel odom、cmd_vel、GT；static / dynamics / long-term changes / combined	真实世界主实验
JRDB	极端人群和遮挡压力测试	360° RGB + LiDAR；大量行人 tracking / panoptic；可出现高拥挤场景	动态鲁棒性 stress test
Dynamic Replica	光流、遮挡、跨帧对应关系辅助实验	stereo、depth、instance mask、foreground/background、optical flow、long-range pixel trajectories	运动补偿 / correspondence ablation

优先级：如果今天开始写代码，优先 ADT sample 跑通 world-slot association；随后用 THUD++ synthetic / Habitat 做可控实验；LILocBench 放到最后作为真实主实验。

3. 从零开始的实施步骤

Step 0 - 复现 DINO-WM 表示范式

先只验证 frozen DINOv2 patch features、trajectory dataloader、latent transition / temporal context 的基本管线。目标不是复现整篇论文，而是确认“RGB -> DINO latent -> 时序模型”没有工程问题。

Step 1 - 先用 ADT 小样本跑通 persistent slot

选一个办公室/公寓 sequence。利用 GT depth + 6DoF camera pose + object_uid，先不做复杂动态检测；验证同一个静态物体在视角变化、部分遮挡、重新出现后，能否一直关联到同一个 memory slot。

Step 2 - 建立统一 episode 数据格式

把 ADT、THUD++、Habitat、LILocBench 都预处理成统一字段。模型只读取统一 dataloader，不直接依赖各数据集原目录。

Step 3 - 实现“frame observation -> world memory”

每帧提取 DINOv2 feature；结合 depth + pose 把区域从 image coordinates 投影到 world coordinates；每个 world slot 保存 latent、语义/实例、几何位置、staticness、last_seen、visibility。

Step 4 - 实现静态 / 动态双记忆更新

动态像素或对象进入短期 memory，不覆盖长期 static slot；只是被遮挡时视为“no new evidence”；多帧持续且几何一致的变化才允许修改长期记忆。

Step 5 - 用 THUD++ / Habitat 做受控消融

固定相机轨迹，逐步增加 pedestrian 数量、遮挡比例、遮挡时长、机器人转角和速度。先用 GT dynamic mask / pose 做 oracle，再替换为估计模块，分清“思路问题”与“前端感知误差”。

Step 6 - ADT 做关联与重现测试

重点测 viewpoint consistency、reappearance consistency、world-slot identity。利用 object_uid 和精确 pose 作为 GT。

Step 7 - LILocBench 做真实主实验

先 mapping sequence 建初始静态记忆，然后分别跑 static、dynamics、long-term changes、long-term changes+dynamics，检验模型是否能区分 temporary occlusion 与 persistent change。

Step 8 - JRDB / Dynamic Replica 做压力与机制分析

JRDB 压人群与遮挡；Dynamic Replica 用 optical flow / long-range trajectories 检查 ego-motion compensation、跨帧对应和动态前景剔除。

4. 推荐的最小可行版本（MVP）

MVP 不要一开始就全堆上

ADT 单 sequence + frozen DINOv2 + GT depth/pose + 简单 object/region slot + static/dynamic GT。先证明“跨视角 persistent memory 比 frame-indexed patch memory 稳定”。这个成立后，再逐步替换 GT、加入透视/平面结构、加入真实动态数据。

5. 统一 episode 格式

建议所有数据在进入模型前转成下面的目录约定：

```
episode_0001/  
  rgb/000000.png ...  
  depth/000000.png ...  
  pose.npy          # T_world_camera 或 T_world_robot  
  action.npy        # velocity / control; 没有则留空  
  semantic/         # 可选  
  instance/         # 可选  
  dynamic_mask/     # GT 或估计  
  timestamps.npy  
  metadata.json
```

6. Memory slot 建议字段

字段	作用
slot_id	世界中的稳定索引，不随 frame patch 编号变化
latent z	由当前可见区域的 DINOv2 feature 池化得到
geometry	3D point / plane / surface support; 至少有世界坐标位置
semantic / instance	墙、门、地板、椅子或 object_uid; 可先用 GT
staticness	属于长期静态结构的置信度
last_seen	最后一次真正被观测到的时间
visibility / occluded	区分“没看到”与“发生变化”
uncertainty	pose、depth、association 的不确定性，用于后续更新门控

7. 论文核心指标

- **Static Memory Retention (SMR):** 动态人/物遮挡以后，静态 slot 的 latent / identity / geometry 保持程度。
- **Dynamic Contamination Rate (DCR):** 动态前景错误写入长期 static memory 的比例；越低越好。
- **Reappearance Consistency:** visible -> occluded -> visible 后，是否重新关联回原 world slot。
- **Viewpoint Consistency:** 机器人平移/转向后，同一墙、门、物体是否保持一致的 world identity。
- **Transient-vs-Persistent Change Accuracy:** 临时遮挡不更新 static；家具长期搬动/结构改变应更新 static。
- **Downstream Navigation / Query:** 记忆是否真正提升目标检索、场景查询或导航，而非只让 latent 更稳定。

8. 关键消融矩阵

实验	Pose	Depth	Dynamic mask	目的
Oracle	GT	GT	GT	先判断 memory 思路本身是否成立
Front-end depth	GT	estimated	GT	隔离深度误差
Front-end motion	GT	estimated	estimated	测试动态分离
Pose noise	noisy/estimated	estimated	estimated	测试转动/漂移鲁棒性
Full system	estimated	estimated	estimated	最终真实设置

9. 实际下载 / 开工顺序

第一周：ADT sample

只下载/使用 sample 或单 sequence。写统一 loader，抽 RGB、depth、trajectory、instance / motion metadata；跑 DINOv2 并实现 world-slot association。

第二周：THUD++ synthetic / emulator

挑一个 corridor/office 类场景，做 0/1/多行人、不同遮挡时长和机器人转向角的受控轨迹。

第三周：LILocBench 小规模真实验证

优先 mapping + static_0 + dynamics_0 + lt_changes_0 + lt_changes_dynamics_0；不必一开始下载全部 600GB 级数据。

之后：JRDB / Dynamic Replica

当主系统工作后再做压力测试；Dynamic Replica 体量很大，只下必要 split / subset。

10. 最终实验故事线

核心实验叙事

DINO-WM 证明 latent dynamics 可用于规划；我们的工作进一步问：当相机持续移动、视角改变并伴随动态人群时，latent 应该如何被组织成“世界中的持续状态”。因此实验必须同时证明三件事：① 跨视角 identity 不丢；② 临时动态不污染长期结构；③ 真实长期变化能够被识别并写回 memory。

官方资料入口（用于后续下载/核对）

DINO-WM: <https://arxiv.org/abs/2411.04983>

THUD++: <https://github.com/jackyzengl/THUD-plus-plus>

THUD Dataset Overview: https://github.com/jackyzengl/THUD_Dataset_Overview

Aria Digital Twin: https://facebookresearch.github.io/projectaria_tools/docs/open_datasets/aria_digital_twin_dataset

ADT Data Format: https://facebookresearch.github.io/projectaria_tools/docs/open_datasets/aria_digital_twin_dataset/data_format

LILocBench / Long-term localization benchmark: https://www.ipb.uni-bonn.de/html/projects/localization_benchmark/

Habitat 3.0: <https://aihabitat.org/habitat3/>

JRDB: <https://jrdb.erc.monash.edu.au/dataset/>

Dynamic Replica: https://github.com/facebookresearch/dynamic_stereo

资料核对备注

本工作笔记中的数据集定位与字段依据上述官方页面 / 论文核对。DINO-WM 的离线轨迹规模示例：PointMaze 2000 条、轨迹长度 100；Wall 1920 条、长度 50；Push-T 18500 条、长度 100-300。LILocBench 官方将序列明确分为 static、short-term dynamics、long-term changes 与二者结合，并提供 RGB-D、odometry 与 cmd_vel。ADT 官方提供 6DoF trajectory、depth、instance segmentation、object_uid 与 static/dynamic motion metadata。